

Ah, my bad entirely. You mean the actual **Claude Code terminal user interface (TUI)** from Anthropic. [1, 2]

Because that payload is formatted exactly as a **Claude Code Skill file** (complete with the --- YAML front matter name and description block), you save it directly inside your project repository so Claude Code can automatically discover and run it via a slash command. [3, 4, 5, 6]

Here is exactly where to input it:

1. Create a skills directory

Inside your project's root folder where you run the claude command, create a dedicated folder for your custom skill definitions: [3]

```
mkdir -p .claude/skills/
```

(Note: Depending on your exact version configuration, Claude Code detects custom skills saved under .claude/skills/ or a global user directory). [7]

2. Save the code as a Markdown file

Create a new file named code-review.md inside that folder: [3, 5, 8]

```
nano .claude/skills/code-review.md
```

Paste your entire text block—including the --- headers—directly into that file and save it. [3, 5]

3. How to use it in the Claude Code TUI

Once saved, launch your terminal interface: [2, 9]

```
claude
```

Because of the name: code-review parameter in your YAML front matter, Claude Code registers this as a custom slash command. You can trigger the workflow inside the chat panel like this: [4, 5]

```
/code-review main
```

(Where main or a commit SHA satisfies your prompt's step 1 requirement to pin the fixed point).

Would you like to write a quick wrapper template to handle the parallel sub-agent triggers automatically, or are you ready to test the file placement?

- [1] <https://code.claude.com>
- [2] <https://the-lean-e-commerce.github.io>
- [3] <https://www.youtube.com>
- [4] <https://code.claude.com>
- [5] <https://avinashselvam.medium.com>
- [6] <https://www.instagram.com>
- [7] <https://www.reddit.com>
- [8] <https://www.linkedin.com>
- [9] <https://www.quvi.in>